

1. The **next greater element** of some element x in an array is the **first greater** element that is **to the right** of x in the same array.

You are given two **distinct 0-indexed** integer arrays nums1 and nums2 , where nums1 is a subset of nums2 .

For each $0 \leq i < \text{nums1.length}$, find the index j such that $\text{nums1}[i] == \text{nums2}[j]$ and determine the **next greater element** of $\text{nums2}[j]$ in nums2 . If there is no next greater element, then the answer for this query is -1 .

Return an array ans of length nums1.length such that $\text{ans}[i]$ is the **next greater element** as described above.

(greedy, easy)

Solution code:

class Solution:

```
def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:
```

```
    stack = []
```

```
    m = {}
```

```
    for i in nums2[::-1]:
```

```
        while stack and stack[-1] <= i:
```

```
            stack.pop()
```

```
        if stack:
```

```
            m[i] = stack[-1]
```

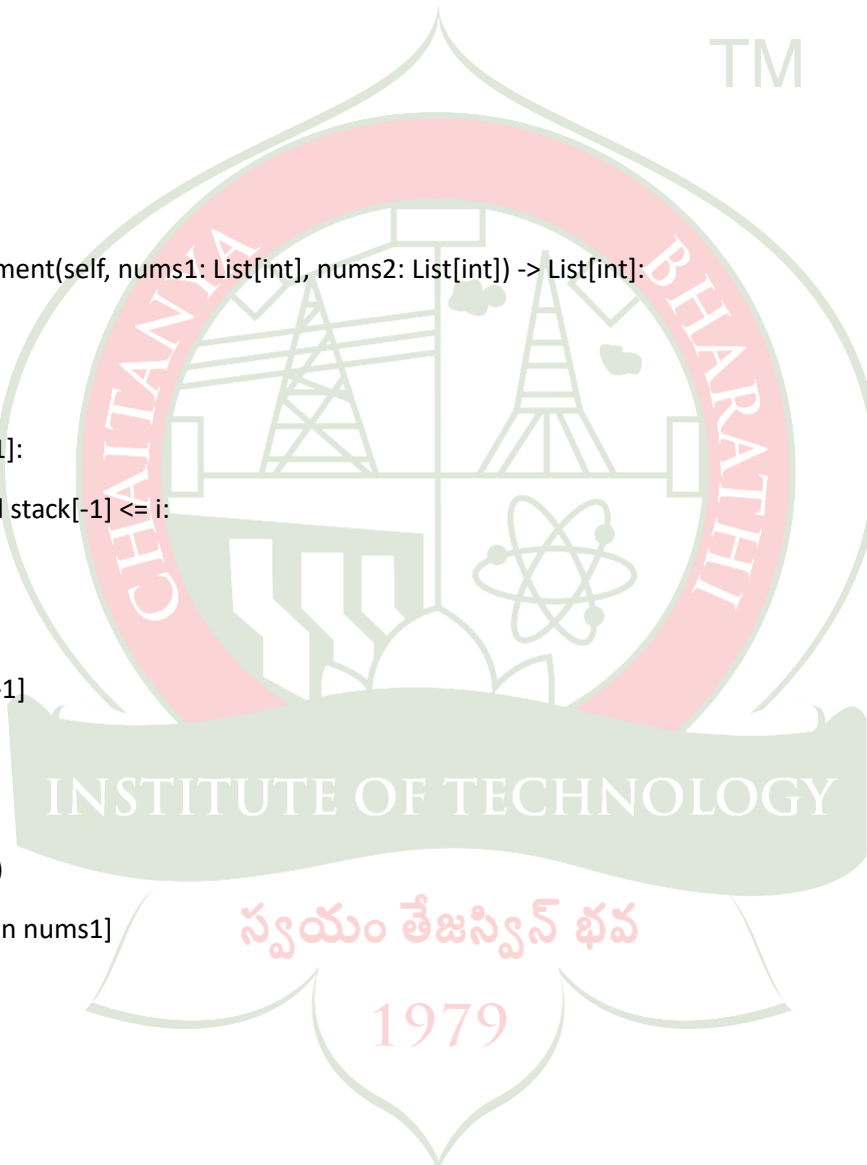
```
        else:
```

```
            m[i] = -1
```

```
        stack.append(i)
```

```
    return [m[j] for j in nums1]
```

Result Analysis:





Description Editorial Solutions Submissions

496. Next Greater Element I

Solved

Easy Topics Companies

The **next greater element** of some element x in an array is the **first greater element** that is **to the right** of x in the same array.

You are given two **distinct 0-indexed** integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each $0 \leq i < \text{nums1.length}$, find the index j such that `nums1[i] == nums2[j]` and determine the **next greater element** of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return an array `ans` of length `nums1.length` such that `ans[i]` is the **next greater element** as described above.

Example 1:

Input: `nums1 = [4,1,2]`, `nums2 = [1,3,4,2]`

Output: `[-1,3,-1]`

8.7K 236 175 Online

Code

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`nums1 =`

`[4,1,2]`

`nums2 =`

`[1,3,4,2]`

Output

`[-1,3,-1]`

Expected

`[-1,3,-1]`

Time Complexity: $O(N + M)$

- Iterating through `nums2` takes $O(N)$ using a monotonic stack.
- Constructing the result for `nums1` takes $O(M)$ using dictionary lookups.

Space Complexity: $O(N)$

- The stack and hashmap store at most N elements from `nums2`.

Conclusion:

The problem of finding the next greater element in `nums1` based on `nums2` is efficiently solved using a **monotonic decreasing stack**. Instead of using a brute-force approach ($O(N * M)$), we preprocess `nums2` in $O(N)$ time to store the next greater elements in a hashmap. This allows us to retrieve results for `nums1` in $O(M)$ time. The overall **time complexity is $O(N + M)$** , and the **space complexity is $O(N)$** due to the stack and hashmap. This approach ensures an optimal and efficient solution.

2. Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return *the next greater number for every element in `nums`*.

The **next greater number** of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

(greedy, medium)

Solution Code:

class Solution:

```
def nextGreaterElements(self, nums: List[int]) -> List[int]:
```

```
    n = len(nums)
```

```
    nge = [-1] * n
```

```
    stack = []
```

```
    for i in range(2*n-1, -1, -1):
```

```
        while stack and stack[-1] <= nums[i % n]:
```

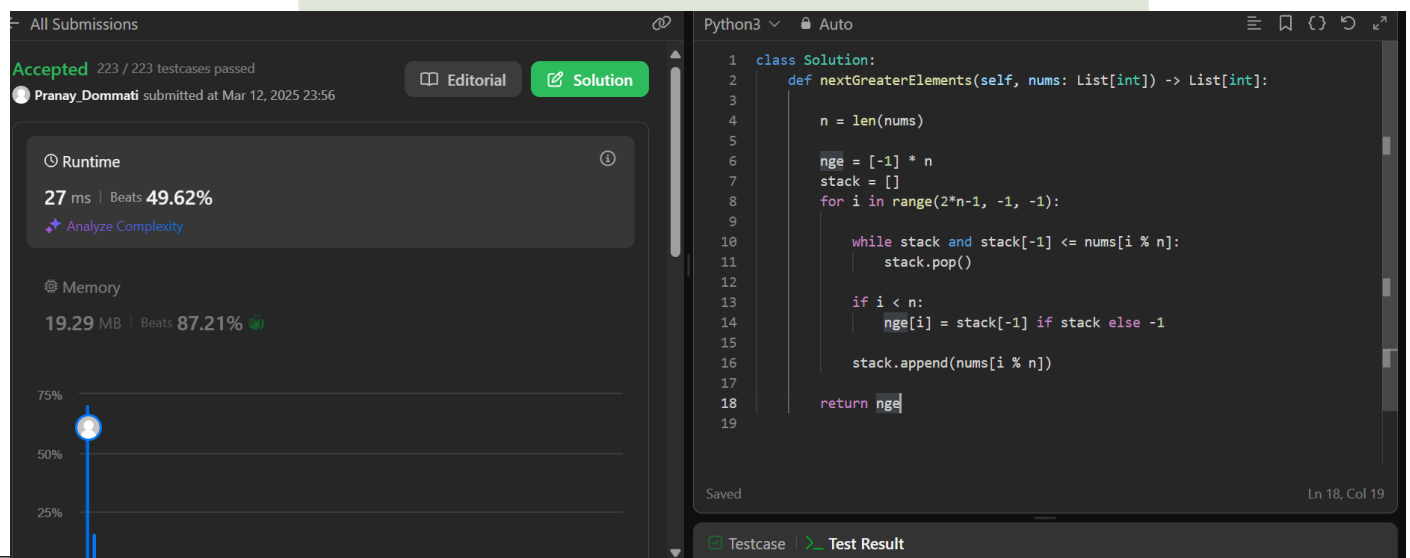
```
            stack.pop()
```

```
        if i < n:
```

```
            nge[i] = stack[-1] if stack else -1
```

```
        stack.append(nums[i % n])
```

```
    return nge
```

Result Analysis:

503. Next Greater Element II

Solved ✓

Medium Topics Companies

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return the **next greater number** for every element in `nums`.

The **next greater number** of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

Example 1:

Input: `nums = [1,2,1]`
Output: `[2,-1,2]`
Explanation: The first 1's next greater number is 2;
The number 2 can't find next greater number.
The second 1's next greater number needs to search circularly, which is also 2.

8.4K 56 82 Online

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`nums =`
`[1,2,1]`

Output

`[2,-1,2]`

Expected

`[2,-1,2]`

Contribute a testcase

503. Next Greater Element II

Solved ✓

Medium Topics Companies

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return the **next greater number** for every element in `nums`.

The **next greater number** of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

Example 1:

Input: `nums = [1,2,1]`
Output: `[2,-1,2]`
Explanation: The first 1's next greater number is 2;
The number 2 can't find next greater number.
The second 1's next greater number needs to search circularly, which is also 2.

8.4K 56 82 Online

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`nums =`
`[1,2,3,4,3]`

Output

`[2,3,4,-1,4]`

Expected

`[2,3,4,-1,4]`

Contribute a testcase

Time Complexity: $O(N)$

- Each element is pushed and popped from the stack **at most once**, leading to an **$O(N)$** overall complexity.
- The loop runs **$2N$** times, but each operation is still amortized **$O(1)$** per element.

Space Complexity: $O(N)$

- The stack stores at most **N** elements, and the nge array also takes **$O(N)$** space.

Conclusion:

The problem of finding the next greater element in a circular array is efficiently solved using a **monotonic decreasing stack**. By iterating **twice** over the array (using indices modulo n), we ensure that all elements have a valid next greater element even in a circular manner. The **time complexity is $O(N)$** since each element is pushed and popped at most once, and the **space complexity is $O(N)$** due to the stack and output array. This approach optimally finds the next greater elements while avoiding redundant comparisons.

3.You are given a **0-indexed** array of non-negative integers `nums`. For each integer in `nums`, you must find its respective **second greater** integer.

The **second greater** integer of `nums[i]` is `nums[j]` such that:

- $j > i$
- `nums[j] > nums[i]`
- There exists **exactly one** index k such that `nums[k] > nums[i]` and $i < k < j$.

If there is no such `nums[j]`, the second greater integer is considered to be -1.

- For example, in the array `[1, 2, 4, 3]`, the second greater integer of 1 is 4, 2 is 3, and that of 3 and 4 is -1.

Return an integer array `answer`, where `answer[i]` is the second greater integer of `nums[i]`.

(greedy, hard)

Solution Code:

class Solution:

```
def secondGreaterElement(self, nums: List[int]) -> List[int]:
```

```
    n = len(nums)
```

```
    result = [-1] * n
```

```
    stack1 = []
```

```
    stack2 = []
```

```
    for i, num in enumerate(nums):
```

```
        while stack2 and nums[stack2[-1]] < num:
```

```
result[stack2.pop()] = num
```

```
temp = []
```

```
while stack1 and nums[stack1[-1]] < num:
```

```
    temp.append(stack1.pop())
```

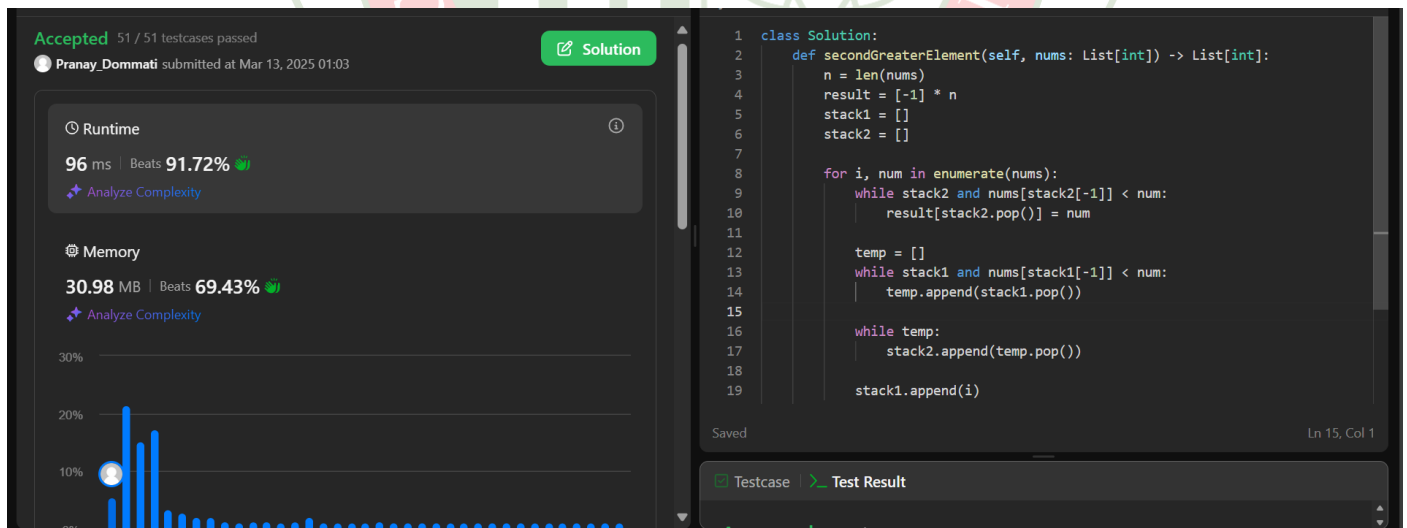
```
while temp:
```

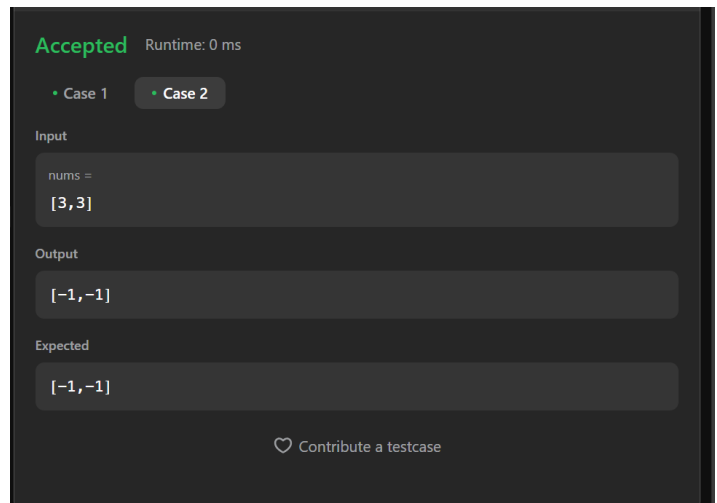
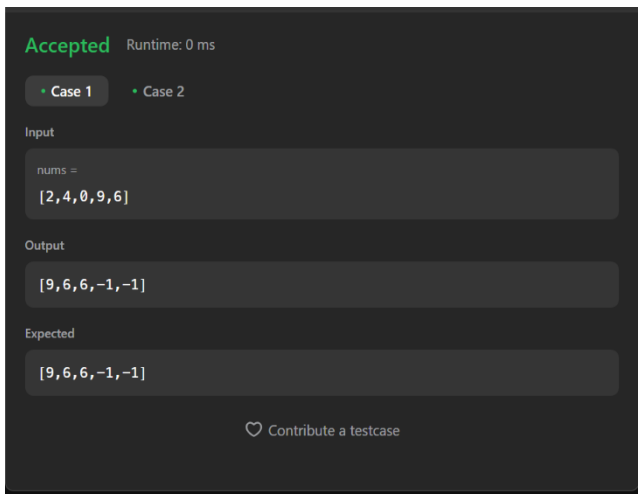
```
    stack2.append(temp.pop())
```

```
stack1.append(i)
```

```
return result
```

Result Analysis:



**Time Complexity:**

- We iterate through nums once ($O(n)$).
- Each element is pushed and popped from stack1 and stack2 at most once ($O(n)$).
- Overall, the time complexity is $O(n)$.

Space Complexity:

- result array takes $O(n)$ space.
- Two stacks stack1 and stack2 together store at most $O(n)$ elements.
- Therefore, the space complexity is $O(n)$.

Conclusion:

This approach efficiently finds the second greater element using two monotonic stacks.

- stack1 helps track next greater elements.
- stack2 helps find the second greater element.
- By ensuring each element is pushed and popped only once, the algorithm runs in $O(n)$ time with $O(n)$ space.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

(dynamic programming, easy)

Solution Code:

class Solution:

```
def climbStairs(self, n: int) -> int:
```

```
    if n == 0 or n == 1:
```

```
        return 1
```

```
    curr = None
```

```
    prev1 = prev2 = 1
```

```
    for i in range(2, n+1):
```

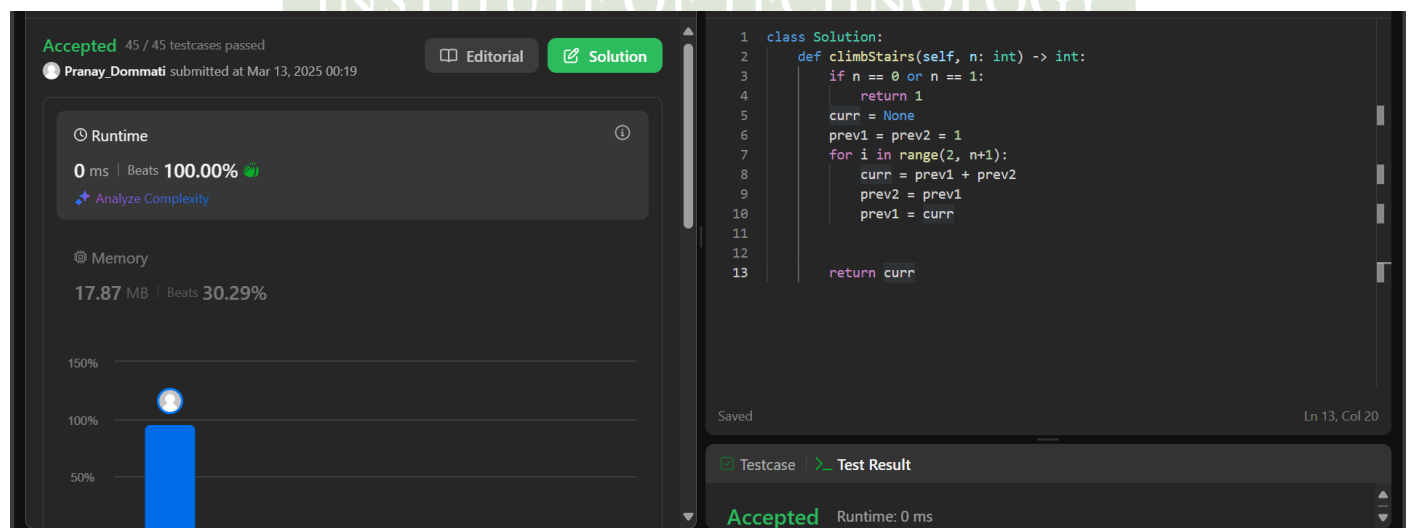
```
        curr = prev1 + prev2
```

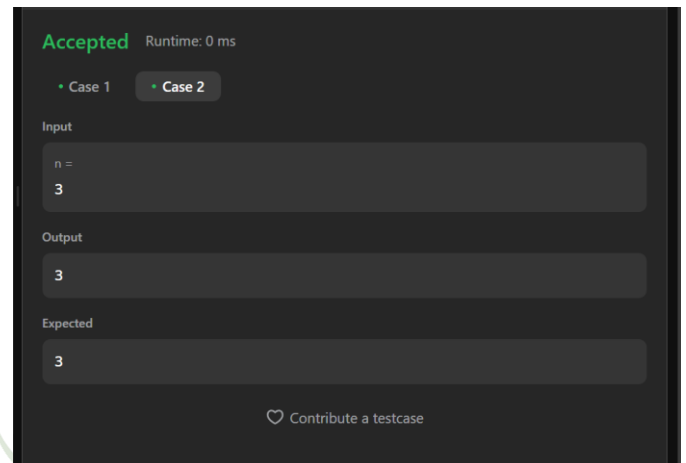
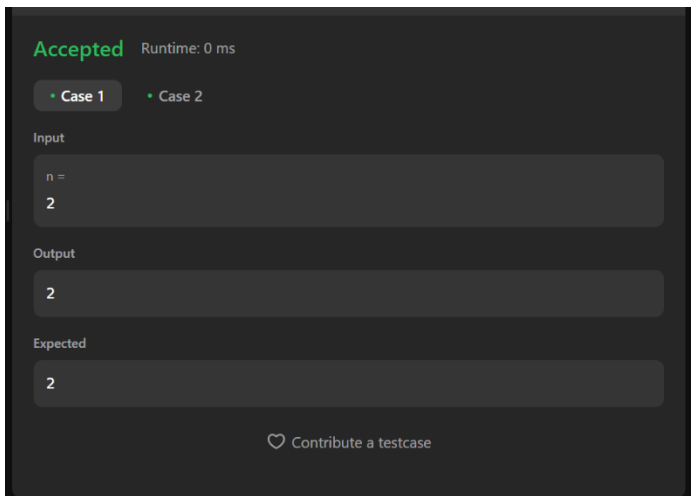
```
        prev2 = prev1
```

```
        prev1 = curr
```

```
    return curr
```

Result Analysis:





Time Complexity: $O(N)$

- A single loop runs from 2 to n , leading to **$O(N)$ time complexity.**

Space Complexity: $O(1)$

- Only a few integer variables (curr, prev1, prev2) are used, leading to **$O(1)$ space complexity.**

Conclusion:

This is a **bottom-up dynamic programming** approach that optimally solves the climbing stairs problem using the Fibonacci recurrence relation. It achieves **$O(N)$ time** with **$O(1)$ space**, making it an efficient solution.

5. You are given an array prices where prices[i] is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

(dynamic programming, easy)

Solution Code:

from typing import List

class Solution:

```
def maxProfit(self, prices: List[int]) -> int:
```

```
    if not prices:
```

```
        return 0
```

```
    min_price = prices[0]
```

```
    max_profit = 0
```

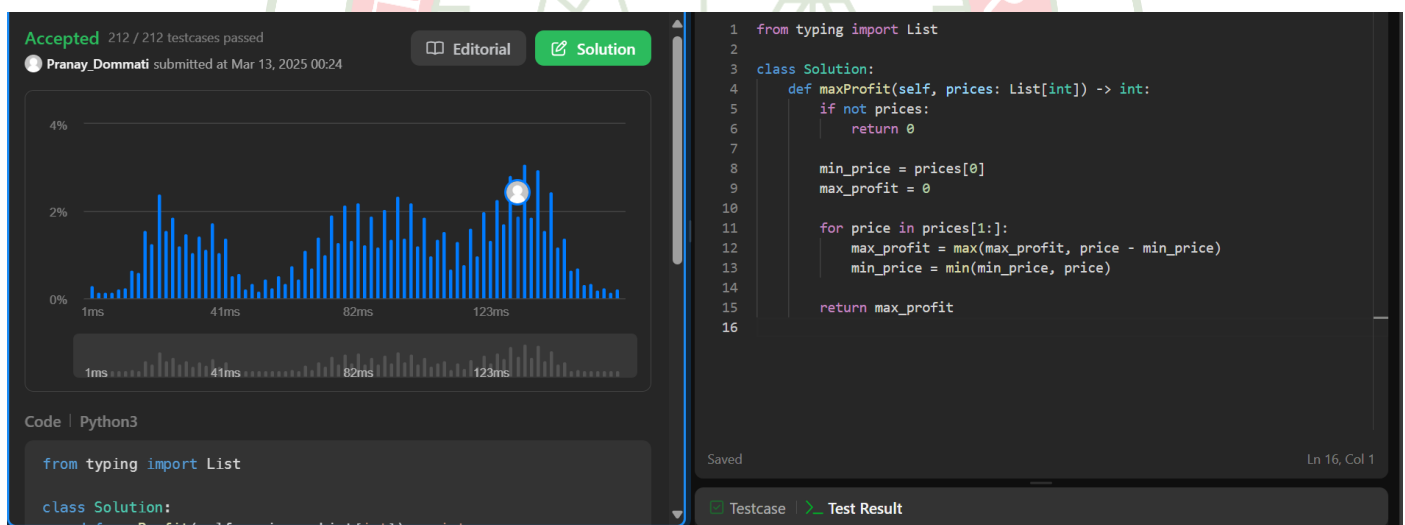
```
    for price in prices[1:]:
```

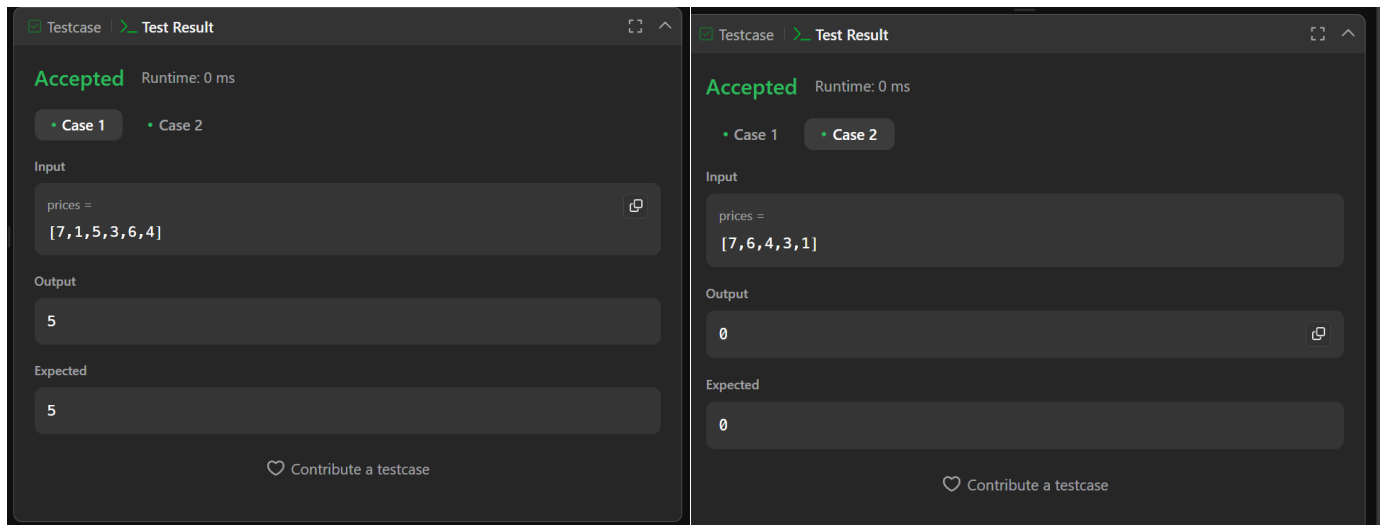
```
        max_profit = max(max_profit, price - min_price)
```

```
        min_price = min(min_price, price)
```

```
    return max_profit
```

Result Analysis:





Time Complexity: $O(N)$

- The loop iterates through the prices list once, making it **$O(N)$** .

Space Complexity: $O(1)$

- Only two variables (min_price and max_profit) are used, so the space complexity is **$O(1)$** .

Conclusion:

This is an **optimized dynamic programming (DP) approach** where we track the **minimum price** seen so far and update the **maximum profit** dynamically. Instead of maintaining a full DP array, it efficiently runs in **$O(N)$ time** and **$O(1)$ space**, making it optimal for solving the problem.

6. Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.
(dynamic programming, medium).

Solution Code:

class Solution:

```
def generateParenthesis(self, n: int) -> List[str]:
```

```
    result = []
```

```
    stack = []
```

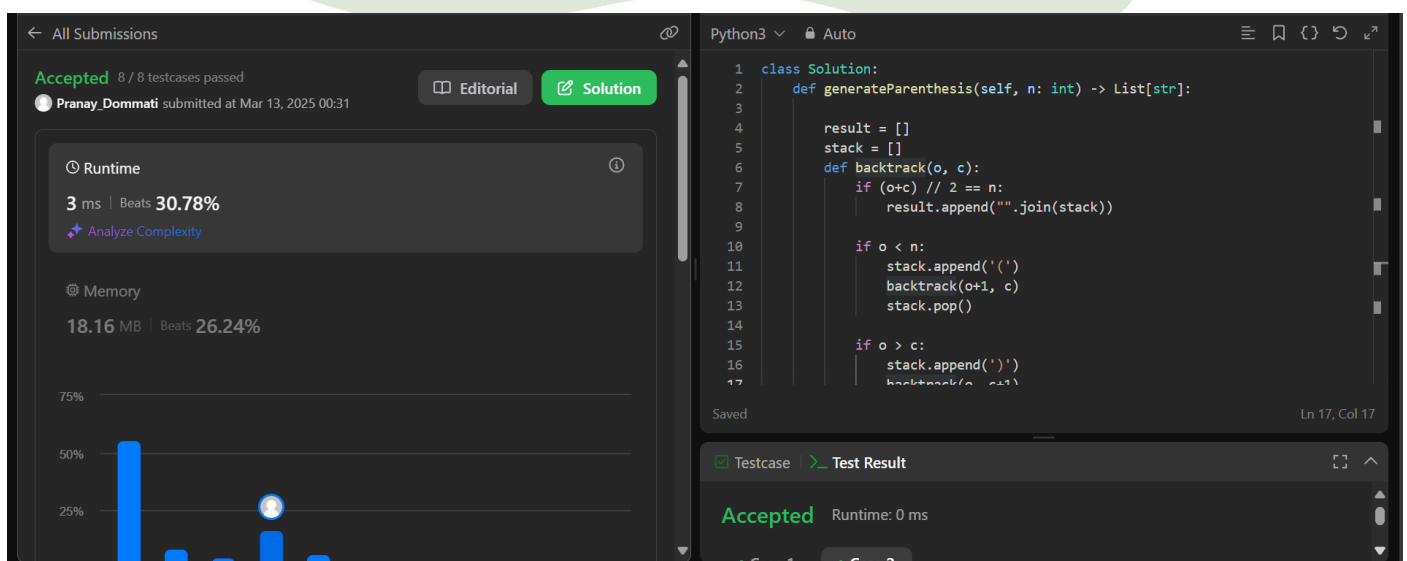
```
    def backtrack(o, c):
```

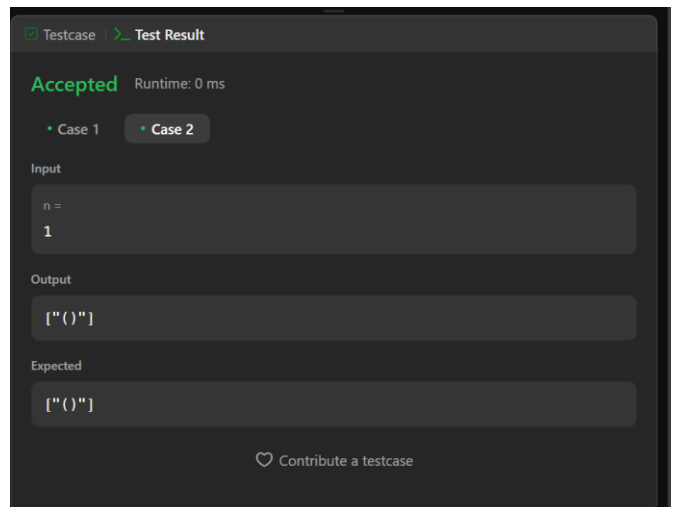
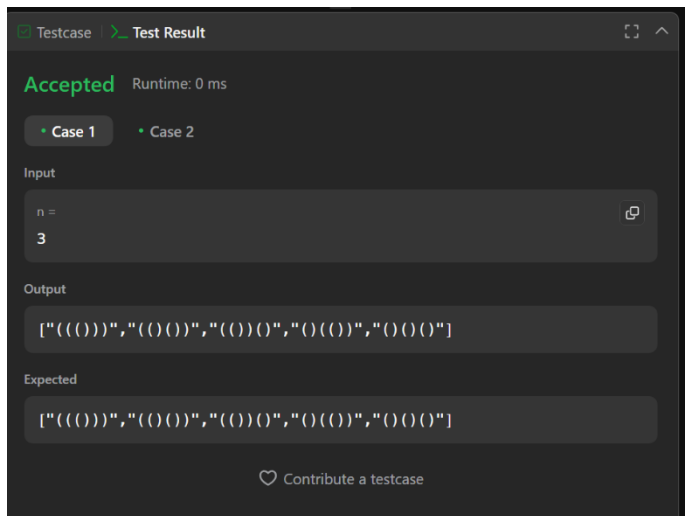
```
if (o+c) // 2 == n:
    result.append(''.join(stack))

if o < n:
    stack.append('(')
    backtrack(o+1, c)
    stack.pop()

if o > c:
    stack.append(')')
    backtrack(o, c+1)
    stack.pop()

backtrack(0, 0)
return result
```

**Result Analysis:**



Time Complexity: $O(2^n)$

- The recursion generates **all valid combinations** of parentheses, leading to an **exponential time complexity of $O(2^n)$** in the worst case.
- The exact count follows the **Catalan number** sequence: $O(4^n / \sqrt{n})$, but it simplifies to $O(2^n)$ for analysis.

Space Complexity: $O(N)$

- The **recursive call stack** has a maximum depth of $O(N)$ due to the n pairs of parentheses.
- The **result list** stores all valid combinations, but since each string is of length $2N$, the storage required is $O(2N \times C_n)$, where C_n is the n th Catalan number.

Conclusion:

This is a **backtracking approach** that efficiently generates all valid parentheses combinations. The algorithm explores **all possible placements** while ensuring valid sequences, pruning invalid ones early. It runs in **$O(2^n)$ time** with **$O(N)$ recursive depth**, making it optimal for small n but expensive for larger values.

7. Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

(dynamic programming, hard)

Solution Code:

class Solution:

def trap(self, height: List[int]) -> int:

n = len(height)

output = []

#prefix_max

prefix_max = []

prefix_max.append(height[0])

for i in range(1, n):

prefix_max.append(max(prefix_max[i-1], height[i]))

#suffix_max

suffix_max = [0] * n

suffix_max[n-1] = height[n-1]

for i in range(n-2, -1, -1):

suffix_max[i] = max(suffix_max[i+1], height[i])

print(suffix_max)

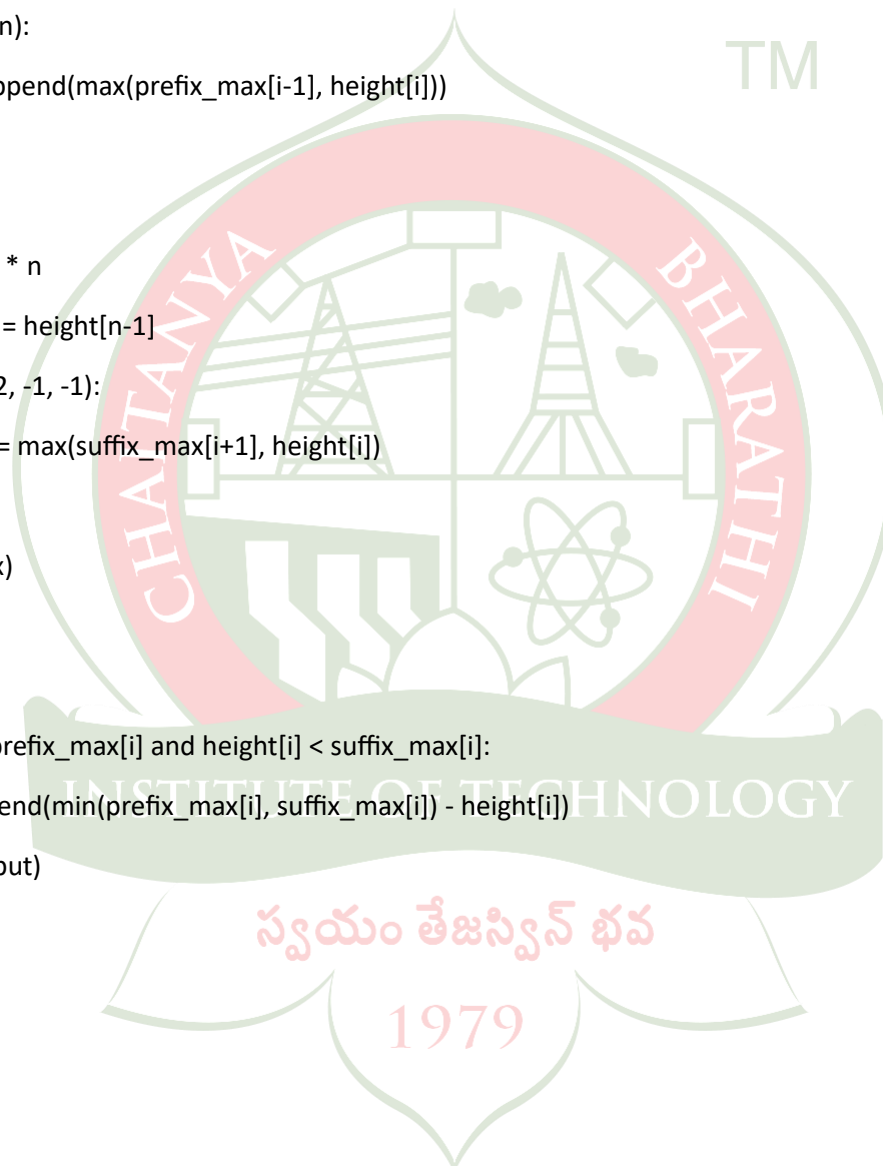
for i in range(n):

if height[i] < prefix_max[i] and height[i] < suffix_max[i]:

output.append(min(prefix_max[i], suffix_max[i]) - height[i])

return sum(output)

Result Analysis:



```
1 class Solution:
2     def trap(self, height: List[int]) -> int:
3         n = len(height)
4         output = []
5
6         #prefix_max
7         prefix_max = []
8         prefix_max.append(height[0])
9         for i in range(1, n):
10             prefix_max.append(max(prefix_max[i-1], height[i]))
11
12         #suffix_max
13         suffix_max = [0] * n
14         suffix_max[n-1] = height[n-1]
15         for i in range(n-2, -1, -1):
16             suffix_max[i] = max(suffix_max[i+1], height[i])
17
18         print(suffix_max)
```

Accepted Runtime: 0 ms

Case 1 Case 2

Input

height =

[0,1,0,2,1,0,1,3,2,1,2,1]

Stdout

[3, 3, 3, 3, 3, 3, 3, 2, 2, 2, 1]

Output

6

Expected

6

Accepted Runtime: 0 ms

Case 1 Case 2

Input

height =

[4,2,0,3,2,5]

Stdout

[5, 5, 5, 5, 5, 5]

Output

9

Expected

9

Time Complexity: $O(N)$

- The algorithm computes `prefix_max` and `suffix_max` in $O(N)$ time.
- The final iteration over `height` to calculate trapped water is also $O(N)$.
- Overall, the time complexity is $O(N) + O(N) + O(N) = O(N)$.

Space Complexity: $O(N)$

- `prefix_max`, `suffix_max`, and `output` all store $O(N)$ elements, leading to $O(N)$ space usage.

Conclusion:

This is a **dynamic programming approach** that precomputes **prefix max** and **suffix max** arrays to efficiently determine trapped rainwater. The solution runs in $O(N)$ time with $O(N)$ space, but can be optimized further to $O(1)$ space using the **two pointer approach**.

8. You are given an array `nums` consisting of **positive** integers and an integer `k`.

Partition the array into two ordered **groups** such that each element is in exactly **one** group. A partition is called great if the **sum** of elements of each group is greater than or equal to `k`.

Return the number of **distinct** great partitions. Since the answer may be too large, return it **modulo** $10^9 + 7$.

Two partitions are considered distinct if some element `nums[i]` is in different groups in the two partitions.

(dynamic programming, hard)

Solution Code:

```
from typing import List

class Solution:
    def countPartitions(self, nums: List[int], k: int) -> int:
```

```
        MOD = 10**9 + 7
```

```
        total_sum = sum(nums)
```

```
        n = len(nums)
```

```
        if total_sum < 2 * k:
```

```
            return 0
```

```
        dp = [0] * k
```

```
        dp[0] = 1
```

```
        for num in nums:
```

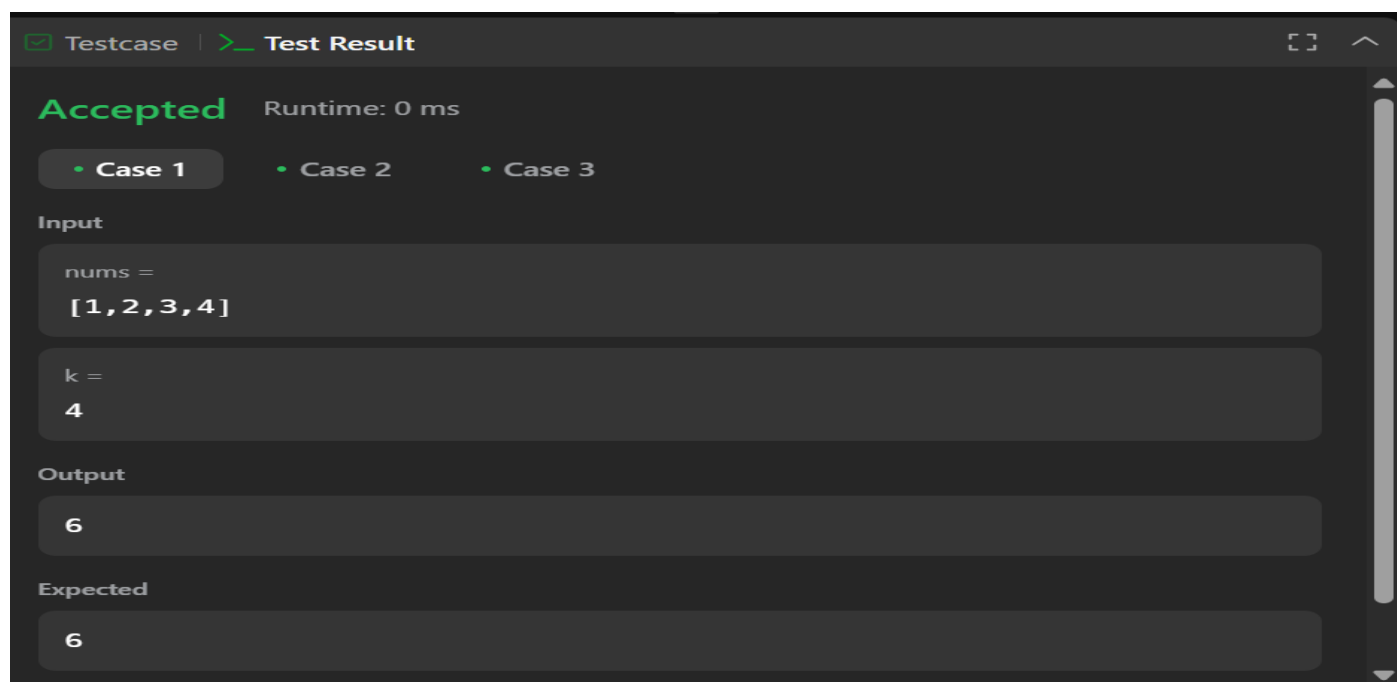
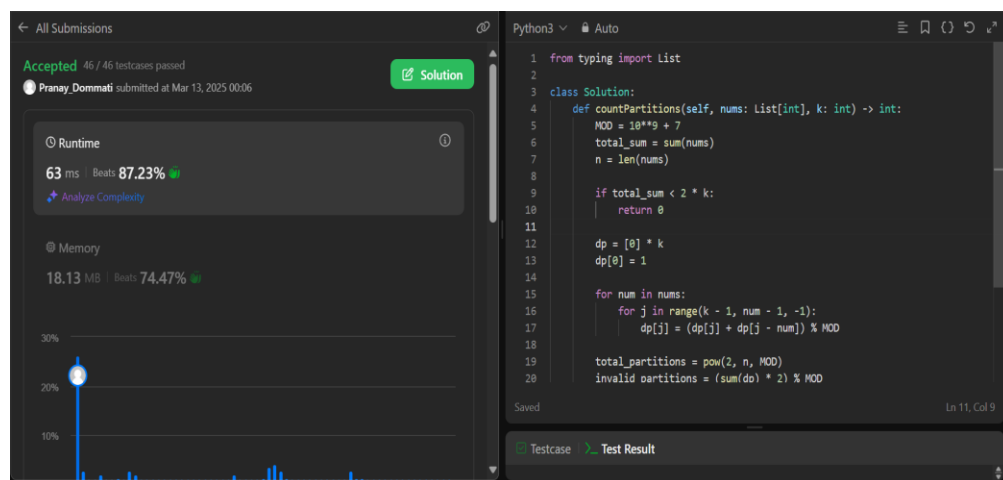
```
            for j in range(k - 1, num - 1, -1):
```

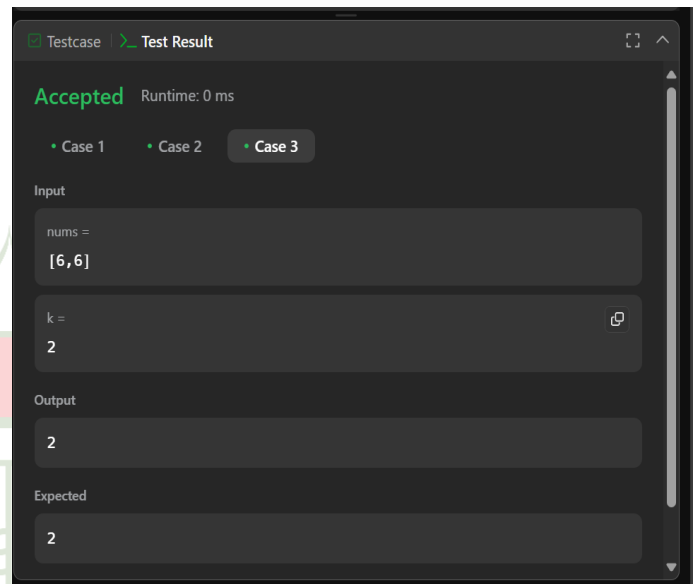
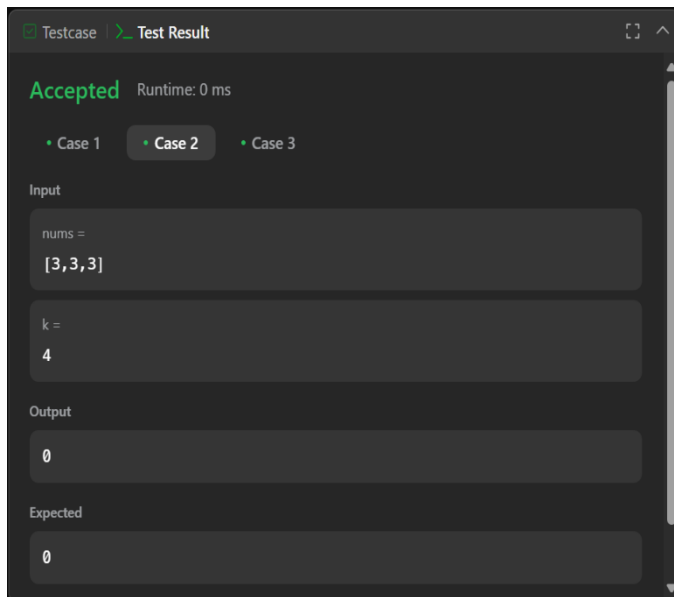
```
                dp[j] = (dp[j] + dp[j - num]) % MOD
```

```
        total_partitions = pow(2, n, MOD)
```

```
        invalid_partitions = (sum(dp) * 2) % MOD
```

```
        return (total_partitions - invalid_partitions) % MOD
```

Result Analysis:



Time Complexity: $O(N * K)$

- The dp array updates for each element in nums, iterating up to k in the worst case.
- This results in $O(N * K)$ complexity.

Space Complexity: $O(K)$

- The dp array of size k is the only extra space used, leading to $O(K)$ space complexity.

Conclusion:

The problem is efficiently solved using **dynamic programming (subset sum DP)**. By computing the number of subsets with sum **less than k**, we determine the count of invalid partitions. The final result is obtained by subtracting these from the total possible partitions (2^n). The approach runs in $O(N * K)$ time and $O(K)$ space, making it optimal given the constraints.